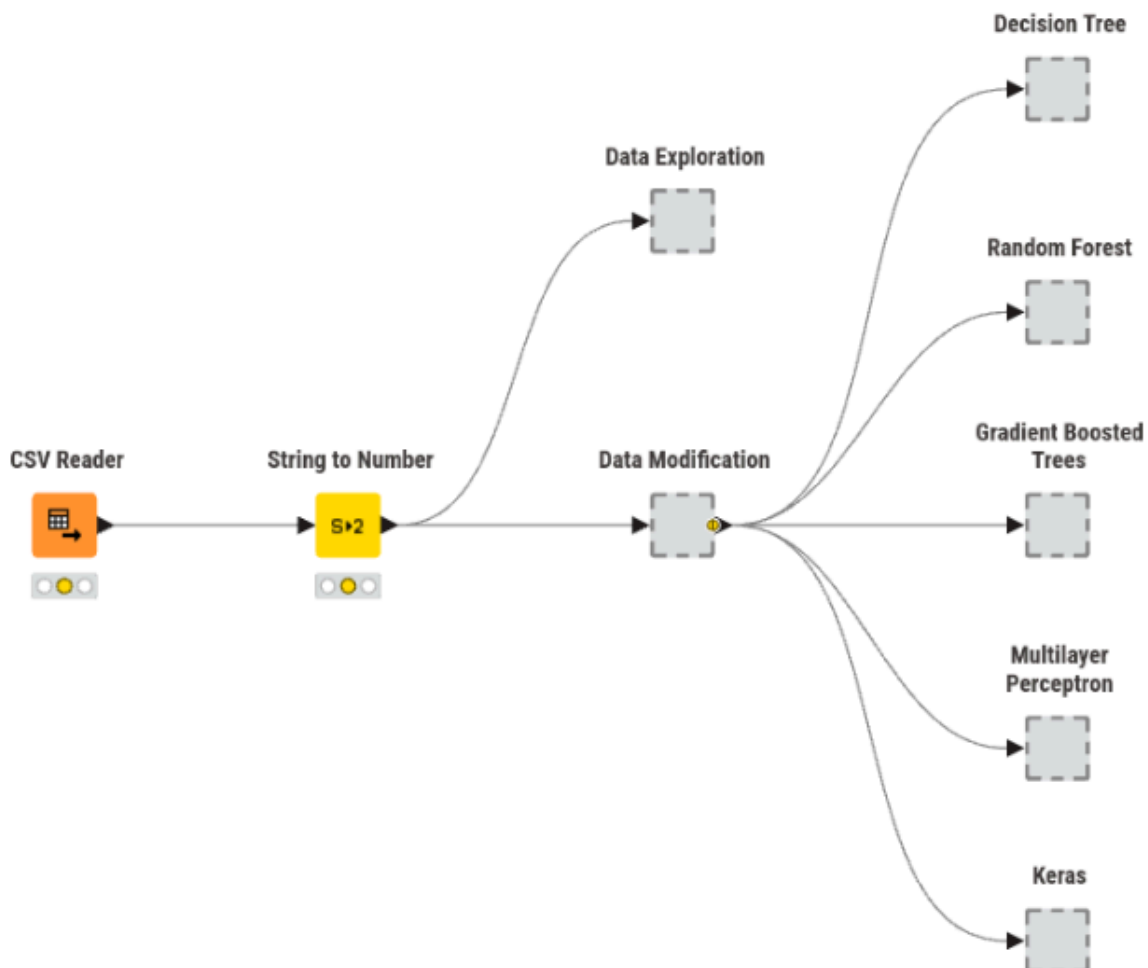


Definição da metodologia

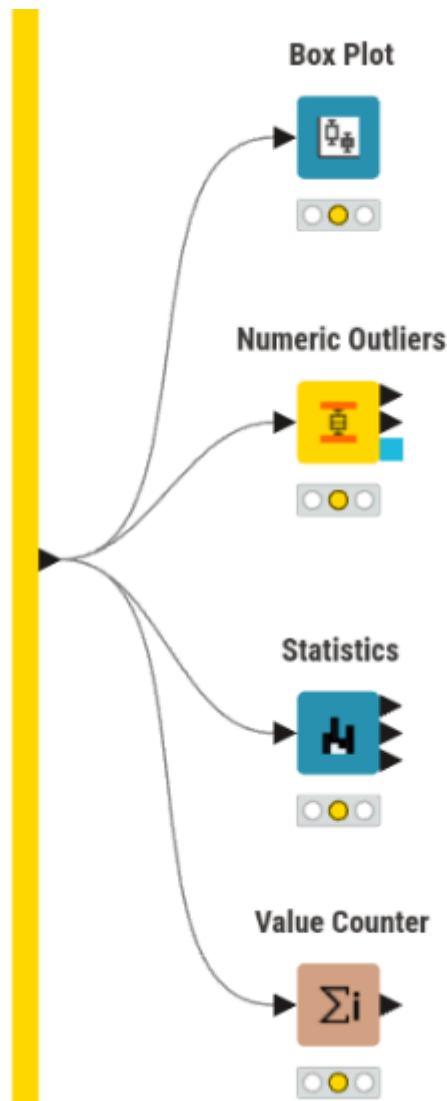
A metodologia de análise de dados que será utilizada neste problema é a SEMMA: Sample, Explore, Modify, Model e Assess. A fase de Sample encontra-se já concretizada na forma do dataset atribuído, pelo que o presente relatório se foca nas fases seguintes. Assim, começou-se por realizar a fase de Explore ou Exploração, cujo objetivo é perceber potenciais tendências e problemas com os dados, para na fase de Modificação se fazerem transformações que visam corrigir os problemas identificados na fase de Exploração. Por fim, aplicam-se as fases de Modelação e Avaliação para cada um dos 5 modelos desenvolvidos para este problema.

Visão geral do workflow desenvolvido no KNIME



De forma geral, o workflow está organizado num conjunto de metanodes que encapsulam os diferentes passos. O nó String to Number, que converte features numéricas que aparecem no dataset em formato string, encontra-se fora de qualquer metanode de forma a propagar os missing values nessas features para a Exploração e para a Modificação simultaneamente. No final, cada um dos 5 modelos recebe o output das transformações dos dados, sendo a Avaliação feita dentro de cada metanode.

Exploração



O primeiro passo da exploração dos dados foi a utilização do nó Statistics para verificar diversas estatísticas, destacando-se as seguintes observações:

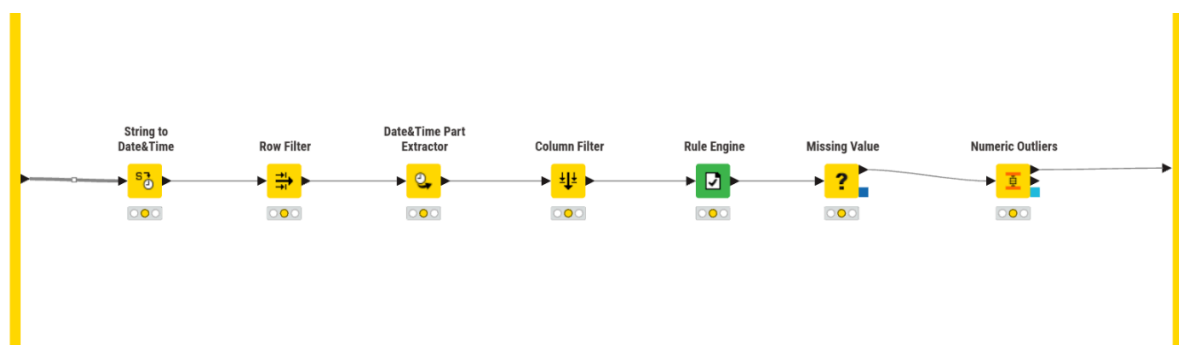
- Precipitation e rain apresentam muitos missing values (~90%)
- Snowfall tem todos os valores a 0 (mean = min = max = 0)
- RowID é redundante
- Todas as restantes features apresentam uma quantidade semelhante de missing values, cerca de 10%. Por isso, deixa de ser viável remover as entradas com missing values, uma vez que, feito isso, o dataset final fica com 1300 entradas (apenas 15% do total). Portanto, foram avaliadas, para cada modelo, diferentes abordagens para imputar os missing values (média e mediana para os numéricos e moda ou valor fixo "Missing" para as strings).

De seguida, foi verificada a presença de outliers, recorrendo tanto a Numeric Outliers para obter resultados numéricos como ao Box Plot para fazer uma exploração mais visual. Desta análise, retirou-se que várias features (como as do vento e as da voltagem) apresentam outliers em quantidade reduzida (<2,5%) e a surface_pressure ligeiramente mais (~5%). No

total, são 1700 as entradas com algum outlier, pelo que a sua remoção não foi considerada, pois perder-se-ia uma boa parte do dataset (cerca de 20%). Portanto, na avaliação dos modelos, os outliers foram ou mantidos ou ajustados para o valor permitido mais próximo, de maneira a avaliar qual o melhor tratamento para cada modelo.

Por fim, foi ainda feita uma análise dos valores das features categóricas com Value Counter, para determinar se existem valores fora do padrão. Concluiu-se que as features Diffuse e Direct Radiation não apresentam valores anormais. Já a target, Consumptions, apresenta, em algumas entradas, valores com erros ortográficos (e.g., Hgh e Lw) ou escritos em português em vez de inglês (e.g., MedioAlto vs Medium-High Consumption).

Transformação



As primeiras transformações aplicadas aos dados centraram-se nos formatos das features, incluindo a conversão das colunas numéricas que se encontravam em string (Medium Voltage e Total), que foi realizada fora deste metanode, conforme descrito anteriormente, e a passagem das datas de string para um formato adequado, além da consequente extração dos campos mais relevantes para este problema (hora, dia da semana e mês). A feature original com a data completa foi removida para evitar introduzir ruído. Ainda, foram detectadas 4 linhas nas quais a data era inválida (e.g., 2025-11-32), pelo que essas entradas foram removidas, por ser uma quantidade muito reduzida.

De seguida, no seguimento da exploração realizada, foram removidas algumas features, sendo elas:

- RowID (redundante)
- precipitation e rain (~90% missing values)
- snowfall (redundante, mean = min = max = 0)

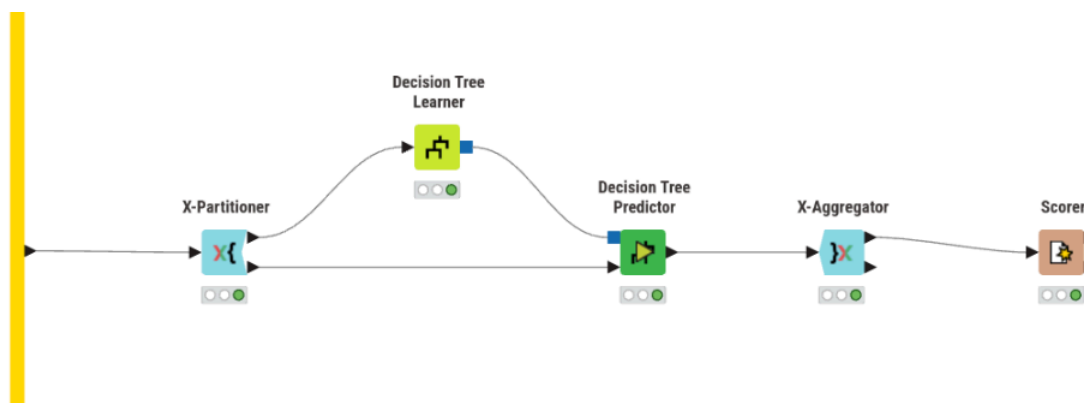
Por fim, de acordo com o que foi possível perceber na exploração, foi aplicada uma Rule Engine para uniformizar o formato dos valores de Consumptions, resultando daqui 5 valores possíveis (Low, MediumLow, Medium, MediumHigh, High).

Modelação e Avaliação

É agora altura de desenvolver modelos para o presente problema. Foram concebidos 5 modelos de classificação, com 3 baseados em árvores e 2 redes neurais. Algumas observações relevantes para as secções seguintes:

1. A partição dos dados para treino e teste foi feita, sempre que possível, com recurso aos nós X-Partitioner e X-Aggregator, uma vez que o uso de cross-validation, embora mais custoso a nível computacional, permite aproveitar a totalidade do dataset e reduzir as chances do modelo sofrer de overfitting;
2. Os nodos de cross-validation foram configurados com uma random seed, de modo a ser possível obter resultados reproduzíveis e sem variabilidade aleatória;
3. Não foram testadas todas as combinações possíveis de transformações de features e configurações dos modelos, pois é uma quantidade impraticável. Por isso, a abordagem adotada foi: testar um conjunto de abordagens e as suas combinações; tirar daí o melhor resultado; realizar o próximo conjunto de testes com a combinação obtida anteriormente. Isto pode fazer com que não sejam detectadas as melhores combinações para cada modelo, mas torna este processo de refinamento mais exequível.

Decision Tree



Tal como mencionado na fase de exploração, os tratamentos de missing values testados foram média e mediana para os valores numéricos e, para os categóricos, moda e valor fixo “Missing”. Já os outliers, tal como dito anteriormente, foram mantidos ou ajustados para o valor permitido mais próximo.

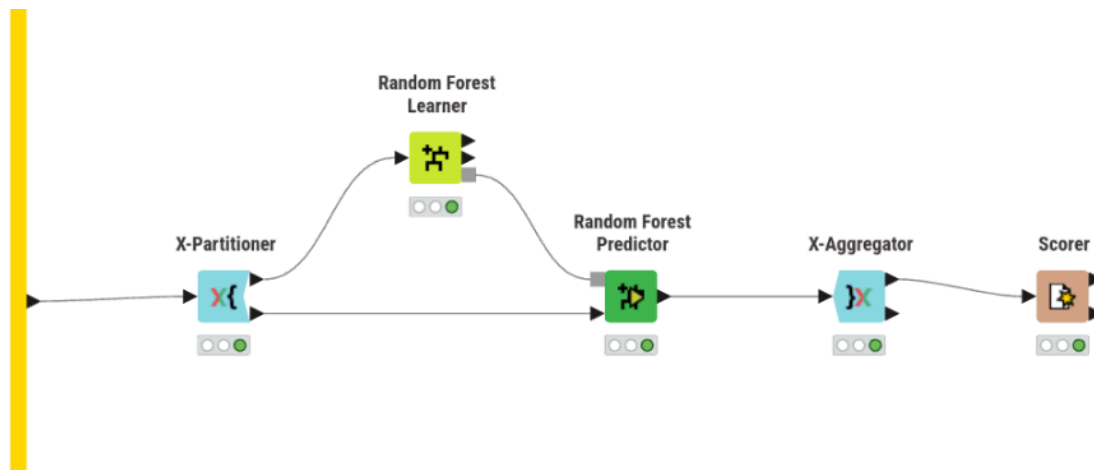
Outliers	Missing values (números)	Missing values (categóricos)	Accuracy	Cohen's Kappa
manter	média	moda	0.884	0.852
manter	média	fixo	0.887	0.855
manter	mediana	moda	0.885	0.853
manter	mediana	fixo	0.888	0.857

ajustar	média	moda	0.885	0.852
ajustar	média	fixo	0.89	0.859
ajustar	mediana	moda	0.885	0.853
ajustar	mediana	fixo	0.889	0.858

Quality Measure	Pruning Method	Min num recs per node	Accuracy	Cohen's Kappa
Gini index	No pruning	2	0.89	0.859
Gini index	No pruning	4	0.897	0.869
Gini index	No pruning	8	0.919	0.897
Gini index	No pruning	12	0.927	0.906
Gini index	MDL	2	0.925	0.904
Gini index	MDL	4	0.925	0.904
Gini index	MDL	8	0.925	0.904
Gini index	MDL	12	0.926	0.905
Gain ratio	No pruning	2	0.888	0.857
Gain ratio	No pruning	4	0.904	0.878
Gain ratio	No pruning	8	0.918	0.895
Gain ratio	No pruning	12	0.922	0.901
Gain ratio	MDL	2	0.919	0.897
Gain ratio	MDL	4	0.919	0.897
Gain ratio	MDL	8	0.921	0.899
Gain ratio	MDL	12	0.921	0.899

Os resultados obtidos mostram que o tratamento de missing values e outliers teve impacto reduzido no desempenho da Decision Tree, verificando-se diferenças muito pequenas entre as diferentes estratégias avaliadas (menos de 1%). Em contrapartida, os hiperparâmetros estruturais da árvore tiveram mais influência no desempenho final. Em particular, o aumento do número de records por nó levou a melhorias consistentes da accuracy e do Cohen's Kappa. Observou-se também que a utilização de pruning não trouxe vantagens relevantes relativamente à ausência de pruning, obtendo resultados ligeiramente inferiores na maioria das configurações, mas reduzindo a variabilidade ao ajustar os outros parâmetros. Quanto à métrica de divisão utilizada, tanto o Gini Index como o Gain Ratio produziram desempenhos semelhantes, embora o Gini Index tenha apresentado, de forma consistente, resultados ligeiramente superiores. O melhor desempenho global foi obtido com Gini Index, sem pruning e com mínimo de 12 registos por nó, atingindo uma accuracy de 0.927 e um Cohen's Kappa de 0.906.

Random Forest



Aqui foram testadas as mesmas combinações de tratamento de missing values e outliers que na Decision Tree.

Outliers	Missing values (números)	Missing values (categóricos)	Accuracy	Cohen's Kappa
manter	média	moda	0.929	0.909
manter	média	fixo	0.929	0.909
manter	mediana	moda	0.93	0.91
manter	mediana	fixo	0.929	0.909
ajustar	média	moda	0.929	0.909
ajustar	média	fixo	0.928	0.908
ajustar	mediana	moda	0.929	0.909
ajustar	mediana	fixo	0.928	0.908

(Com Number of models = 100)

Split criterion	Limit number of levels	Minimum child node size	Accuracy	Cohen's Kappa
Information Gain Ratio	off	off	0.93	0.91
Information Gain Ratio	off	2	0.929	0.909
Information Gain Ratio	off	4	0.927	0.907
Information Gain Ratio	10	off	0.919	0.896

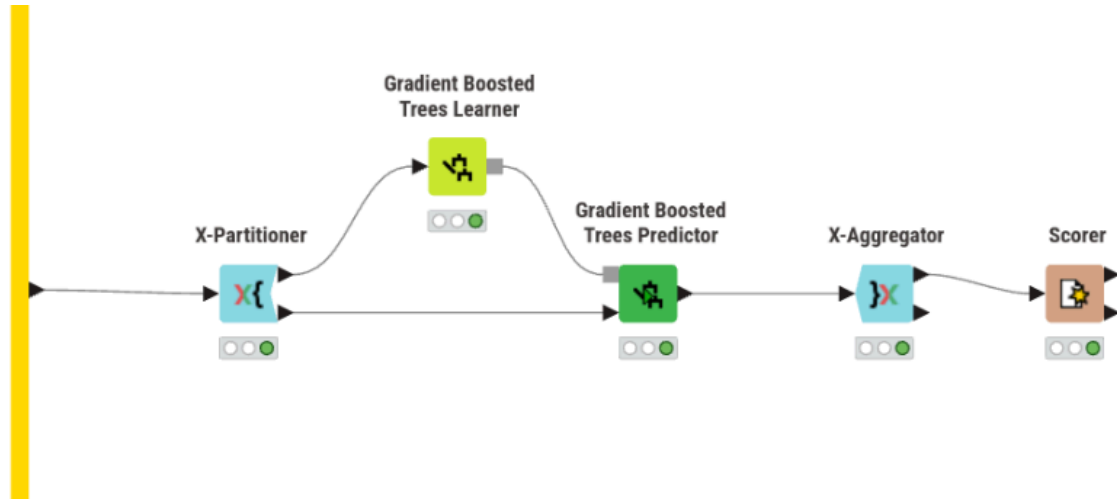
Information Gain Ratio	10	2	0.919	0.896
Information Gain Ratio	10	4	0.918	0.895
Information Gain	off	off	0.929	0.909
Information Gain	off	2	0.928	0.908
Information Gain	off	4	0.927	0.907
Information Gain	10	off	0.922	0.9
Information Gain	10	2	0.921	0.899
Information Gain	10	4	0.921	0.898
Gini	off	off	0.929	0.909
Gini	off	2	0.928	0.907
Gini	off	4	0.927	0.907
Gini	10	off	0.92	0.898
Gini	10	2	0.92	0.897
Gini	10	4	0.921	0.899

Number of models	Accuracy	Cohen's Kappa
50	0.929	0.909
100	0.93	0.91
150	0.93	0.91
200	0.929	0.909
250	0.929	0.909

Tal como observado na Decision Tree, as diferentes estratégias de tratamento de missing values e outliers tiveram impacto praticamente nulo no desempenho da Random Forest. Relativamente aos hiperparâmetros da Random Forest, verificou-se que limitar o número de níveis das árvores provocou uma redução consistente do desempenho. Isto sugere que este problema beneficia de árvores mais profundas, capazes de capturar relações mais complexas entre as features. Também o aumento do parâmetro Minimum child node size levou a pequenas perdas de desempenho. Os diferentes critérios de divisão testados (Information Gain Ratio, Information Gain e Gini) apresentaram desempenhos muito semelhantes, embora o Information Gain Ratio tenha obtido, de forma consistente, os melhores resultados. Finalmente, a avaliação do número de modelos mostrou que aumentar o número de árvores acima de 100 não produziu melhorias relevantes. Isto indica que a Random Forest atinge rapidamente estabilidade neste problema, sendo que ensembles

maiores apenas aumentam o custo computacional sem ganhos significativos de desempenho.

Gradient Boosted Trees



Ao contrário do que foi feito até agora, não foram avaliadas as estratégias de imputação de missing values, uma vez que este modelo dispõe de mecanismos para lidar com este problema. Sendo assim, foram testados estes métodos juntamente com o tratamento de outliers com as configurações padrão, antes de começar a variar os hiperparâmetros.

Outliers	Missing values	Accuracy	Cohen's Kappa
manter	XGBoost	0.928	0.908
manter	Surrogate	0.897	0.868
ajustar	XGBoost	0.928	0.908
ajustar	Surrogate	0.896	0.867

É possível verificar que a configuração com ajuste de outliers tem um resultado igual a não tratar outliers (usando em ambos os casos XGBoost), pelo que se optou por utilizar esta última abordagem, por não introduzir dados artificiais.

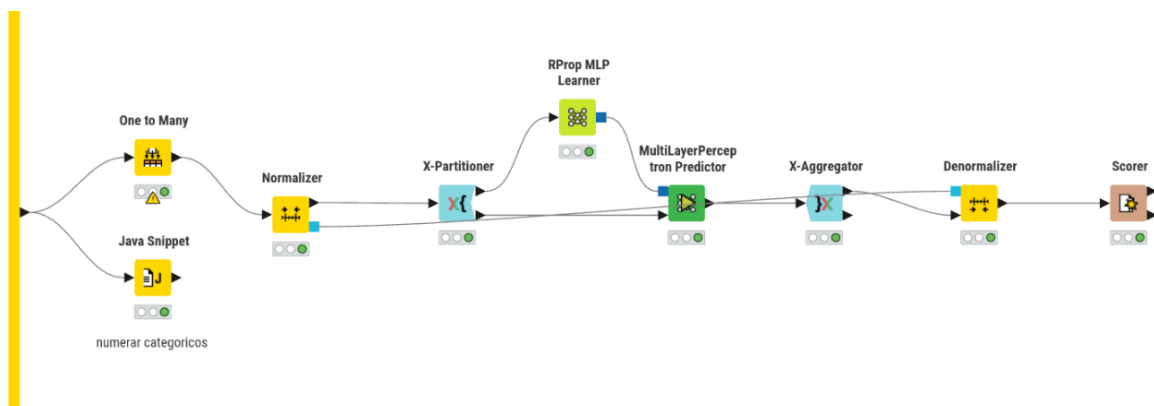
Na avaliação que se segue, a variação do learning rate foi feita em conjunto com o número de modelos, ou seja, diminuindo a learning rate, aumenta-se o número de modelos. Isto foi feito porque, ao diminuir a learning rate, cada árvore individual contribui menos para o resultado final, pelo que é necessário aumentar o total de árvores para compensar.

Learning Rate	Number of Models	Limit number of levels	Row Sampling	Accuracy	Cohen's Kappa
0.1	100	4	off	0.928	0.908
0.1	100	4	0.5	0.929	0.909

0.1	100	4	0.8	0.929	0.909
0.1	100	8	off	0.929	0.909
0.1	100	8	0.5	0.931	0.912
0.1	100	8	0.8	0.932	0.913
0.05	200	4	off	0.928	0.908
0.05	200	4	0.5	0.929	0.909
0.05	200	4	0.8	0.929	0.909
0.05	200	8	off	0.928	0.908
0.05	200	8	0.5	0.932	0.913
0.05	200	8	0.8	0.932	0.912
0.02	400	4	off	0.928	0.908
0.02	400	4	0.5	0.928	0.908
0.02	400	4	0.8	0.928	0.908
0.02	400	8	off	0.93	0.91
0.02	400	8	0.5	0.932	0.913
0.02	400	8	0.8	0.932	0.913

Os resultados mostram que diferentes combinações de learning rate e número de modelos conduzem praticamente ao mesmo desempenho máximo (accuracy = 0.932 e Cohen's kappa = 0.913), sugerindo que, para este problema, o aumento do número de modelos juntamente com a redução da learning rate não produzem ganhos relevantes. Em contrapartida, observou-se que configurações com maior profundidade das árvores e com row sampling ativo tendem a apresentar desempenhos ligeiramente superiores, indicando que estes parâmetros têm maior influência no resultado final do que a combinação learning rate / número de modelos.

Multilayer Perceptron



Para as redes neuronais, uma transformação de dados importante é a normalização (e consequente desnormalização), pelo que foram testados 3 métodos diferentes para a normalização dos dados. No caso do método Min-max, usar o intervalo [0;1] produziu resultados bastante medíocres. Por isso, o intervalo usado foi [-1;1], que melhorou a performance da rede por centrar os dados em zero. Ainda, o método decimal scaling também produziu resultados pouco satisfatórios (accuracy na ordem dos 60%), pelo que não foi avaliado com o mesmo detalhe que foram os outros.

A outra transformação essencial tem a ver com as features categóricas nominais, que não são adequadas para dar como input a uma rede neuronal. Foram testadas duas formas de as tornar apropriadas ao modelo:

- **numerar:** atribuir valores numéricos sequenciais às categorias. Esta forma de tratamento pode não ser 100% adequada porque introduz na feature uma noção de ordem que pode não ser verdadeira/aplicável. No entanto, para este problema, as features afetadas usam classificações como None, Low, Medium e High, pelo que a numeração pode fazer sentido;
- **One to Many:** nó que, para cada categoria de uma feature, cria uma nova feature cujo nome é o nome da original junto com cada valor, e atribui 0 ou 1 conforme o valor da feature naquela entrada. Este método permite tornar features categóricas adequadas para redes neuronais, sem introduzir noções de ordem como a simples numeração.

Um detalhe importante, que difere consoante qual das duas abordagens é utilizada, é quando se faz a normalização. No caso da numeração, a normalização é feita depois, de modo a englobar os novos valores criados. Assim, evita-se que a feature fique com uma escala diferente das restantes. Já no caso de se usar o One to Many, a normalização é feita antes, pois os valores já estão numa escala consistente (0 ou 1) e assim não se destrói a interpretação binária, que é precisamente o objetivo desta abordagem.

Considerando tudo isto, começou-se por avaliar todas as combinações destas configurações, uma vez que são dependentes umas das outras. Por exemplo, os melhores tratamentos de missing values e outliers com Z-score podem ser os piores para Min-max, e portanto a avaliação não estaria correta. O único método de normalização não avaliado com a mesma profundidade que os outros dois foi o Decimal Scaling, porque fez o modelo obter uma accuracy de ~60% com algumas combinações variadas dos restantes parâmetros em avaliação. Assim, foi possível reduzir ligeiramente o espaço de procura (48 para 32 combinações).

Método de normalização	Variáveis categóricas	Outliers	Missing values (números)	Missing values (categóricos)	Accuracy	Cohen's Kappa
Min-max	numerar	manter	média	moda	0.839	0.793
Min-max	numerar	manter	média	fixo	0.845	0.802
Min-max	numerar	manter	mediana	moda	0.841	0.796
Min-max	numerar	manter	mediana	fixo	0.84	0.795
Min-max	numerar	ajustar	média	moda	0.847	0.803

Min-max	numerar	ajustar	média	fixo	0.838	0.792
Min-max	numerar	ajustar	mediana	moda	0.858	0.818
Min-max	numerar	ajustar	mediana	fixo	0.864	0.826
Min-max	One to Many	manter	média	moda	0.855	0.814
Min-max	One to Many	manter	média	fixo	0.848	0.805
Min-max	One to Many	manter	mediana	moda	0.857	0.817
Min-max	One to Many	manter	mediana	fixo	0.841	0.796
Min-max	One to Many	ajustar	média	moda	0.857	0.817
Min-max	One to Many	ajustar	média	fixo	0.856	0.816
Min-max	One to Many	ajustar	mediana	moda	0.86	0.821
Min-max	One to Many	ajustar	mediana	fixo	0.853	0.812
Z-score	numerar	manter	média	moda	0.862	0.823
Z-score	numerar	manter	média	fixo	0.862	0.823
Z-score	numerar	manter	mediana	moda	0.86	0.82
Z-score	numerar	manter	mediana	fixo	0.86	0.821
Z-score	numerar	ajustar	média	moda	0.864	0.825
Z-score	numerar	ajustar	média	fixo	0.858	0.817
Z-score	numerar	ajustar	mediana	moda	0.867	0.83
Z-score	numerar	ajustar	mediana	fixo	0.856	0.815
Z-score	One to Many	manter	média	moda	0.882	0.849
Z-score	One to Many	manter	média	fixo	0.857	0.817
Z-score	One to Many	manter	mediana	moda	0.876	0.841
Z-score	One to Many	manter	mediana	fixo	0.862	0.823
Z-score	One to Many	ajustar	média	moda	0.878	0.844
Z-score	One to Many	ajustar	média	fixo	0.858	0.819
Z-score	One to Many	ajustar	mediana	moda	0.886	0.854
Z-score	One to Many	ajustar	mediana	fixo	0.858	0.818

Estas combinações foram testadas com hiperparâmetros default do modelo (100 iterações, 1 hidden layer e 10 neurónios por layer).

Nº iterations	Nº hidden layers	Nº hidden neurons per layer	Accuracy	Cohen's Kappa
100	1	10	0.886	0.854

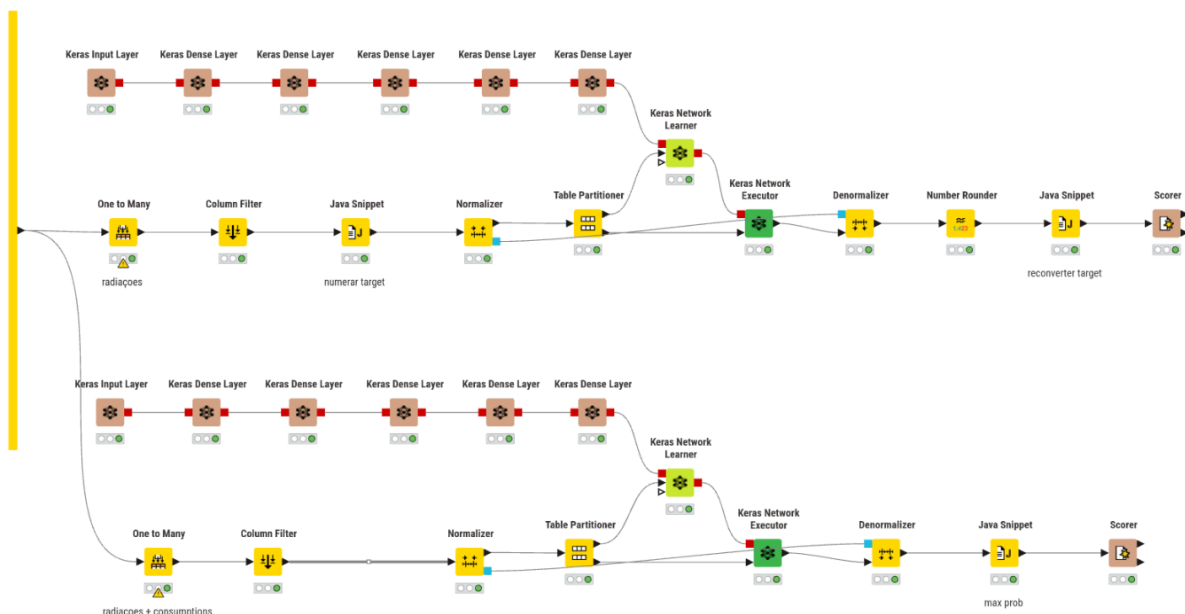
100	1	20	0.885	0.853
100	1	30	0.882	0.848
100	1	40	0.885	0.853
100	2	10	0.843	0.798
100	2	20	0.87	0.833
100	2	30	0.879	0.845
100	2	40	0.867	0.83
100	3	10	0.829	0.78
100	3	20	0.83	0.782
100	3	30	0.841	0.797
100	3	40	0.856	0.816
200	1	10	0.9	0.873
200	1	20	0.899	0.871
200	1	30	0.896	0.867
200	1	40	0.893	0.863
200	2	10	0.866	0.828
200	2	20	0.889	0.859
200	2	30	0.879	0.845
200	2	40	0.869	0.833
200	3	10	0.857	0.817
200	3	20	0.846	0.802
200	3	30	0.862	0.823
200	3	40	0.86	0.821
300	1	10	0.903	0.875
300	1	20	0.9	0.872
300	1	30	0.895	0.865
300	1	40	0.888	0.856
300	2	10	0.884	0.851
300	2	20	0.889	0.858
300	2	30	0.877	0.843
300	2	40	0.865	0.828
300	3	10	0.872	0.837

300	3	20	0.853	0.812
300	3	30	0.862	0.823
300	3	40	0.858	0.819

Os resultados obtidos mostram que o desempenho do Multilayer Perceptron depende mais das transformações aplicadas aos dados do que os modelos anteriores, com ênfase para a normalização e a representação das variáveis categóricas. Dentre os métodos testados, a normalização Z-score apresentou consistentemente melhores resultados do que Min-max e Decimal Scaling. Relativamente às variáveis categóricas, a abordagem One to Many revelou-se superior à simples numeração das categorias neste problema.

Quanto à arquitetura da rede, verificou-se que aumentar o número de iterações melhorou consistentemente os resultados, indicando que o modelo beneficiou de mais tempo de treino. No entanto, o aumento do número de hidden layers e neurónios nem sempre conduziu a melhorias. Neste problema, os resultados sugerem que arquiteturas relativamente simples já conseguem capturar os padrões mais relevantes presentes no dataset. Assim, o aumento da profundidade e do número de neurónios introduziu maior complexidade de otimização sem acrescentar capacidade útil de generalização em relação aos dados de treino, levando em vários casos a uma degradação do desempenho no conjunto de teste.

Keras



O último modelo avaliado foi mais uma rede neuronal, desta vez usando a biblioteca Keras, que possibilita explorar arquiteturas mais flexíveis e diferentes métodos de otimização. Portanto, as transformações dos dados avaliadas para o MLP não foram reavaliadas, tendo-se optado por manter a melhor combinação encontrada (normalização Z-score, One to Many para as features categóricas, ajuste de outliers, mediana para os missing values

numéricos e moda para os missing values categóricos). Portanto, agora o foco passa a ser a definição da arquitetura da rede e do processo de treino.

Antes de prosseguir para a arquitetura da rede, é necessário efetuar algumas transformações nos dados para ser possível aplicar a rede ao problema de classificação. Em particular, é preciso transformar a target Consumptions, pois ao contrário do RProp MLP, estas redes neuronais só recebem e devolvem valores numéricos como resultado. Desta forma, à semelhança do que aconteceu com o modelo anterior, foram testadas duas formas de fazer esta transformação:

- **One to Many:** cria-se uma coluna para cada possível valor da feature original, ou seja, cria-se um conjunto de booleanos. Para o output da rede, usam-se 5 neurónios, obtendo-se 5 valores de 0 a 1 que somam 1, ou seja, probabilidades. Depois é só escolher o maior para se fazer a previsão daquela entrada. Embora este formato de 0s ou 1s já seja aceite pela rede, a noção lógica destes valores é perdida, pelo que esta poderá não ser a melhor abordagem;
- **numerar:** numerar de 1 a 5 os possíveis valores de Consumptions. Este tratamento é adequado porque os valores da target já têm uma ordem própria (Low < MediumLow < Medium < MediumHigh < High), pelo que o significado da numeração continua a fazer sentido para a rede, ao contrário dos booleanos criados com o método anterior. Aqui, a camada de output passa a ter apenas 1 neurónio. O valor gerado pela rede é limitado ao intervalo válido e depois arredondado (e.g., 6.1 => 5 e 0.2 => 1), sendo por fim feita a conversão inversa para o valor em string correspondente.

Como configuração inicial, usou-se uma hidden layer com 16 neurónios e função de ativação ReLU (tal como em todas as futuras hidden layers adicionadas), além da camada de output com 1 ou 5 neurónios, conforme a transformação aplicada à target. Quanto às configurações desta última, no caso do One to Many, a loss function usada é a Categorical Cross Entropy, juntamente com a função de ativação Softmax, que transforma o output nas probabilidades mencionadas anteriormente. Já no caso de se utilizar a numeração, a loss function usada é a Mean Absolute Error e a função de ativação é a Linear, pois não se devem fazer modificações ao output da rede, de maneira a ser possível arredondar o valor produzido. Por fim, o otimizador utilizado em ambas as situações foi o Adam, com os parâmetros padrão.

Depois da configuração inicial, foram testadas outras arquiteturas com mais hidden layers e também neurónios, mantendo o “afunilamento”, ou seja, reduzindo o número de neurónios em cada layer até chegar ao output. Foram utilizados números de neurónios múltiplos de 2 por conveniência experimental e alinhamento com práticas comuns de implementação. Após ser determinada a melhor arquitetura da rede e método de conversão da target, foram experimentados diferentes números de epochs, valor que se fixou inicialmente em 5 como compromisso entre capacidade de aprendizagem e custo computacional.

Nota: o uso de cross-validation tornou-se computacionalmente inviável com este modelo, cuja duração do treino se começa a tornar excessivamente longa numa máquina casual, especialmente com mais hidden layers e neurónios. Por isso, foi usado um Table Partitioner, fazendo uma partição de 70/30 com uma random seed fixa.

One to Many:

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
------------------	----------	----------	---------------

1	16	0.576	0.457
2	16 8	0.639	0.537
2	32 16	0.724	0.647
3	32 16 8	0.752	0.682
3	64 32 16	0.777	0.715
4	128 64 32 16	0.779	0.718

Numerar:

Nº hidden layers	Nº units	Accuracy	Cohen's Kappa
1	16	0.308	0.116
2	16 8	0.297	0.102
2	32 16	0.377	0.201
3	32 16 8	0.36	0.182
3	64 32 16	0.434	0.272
4	128 64 32 16	0.513	0.371

Estratégia	Nº epochs	Accuracy	Cohen's Kappa
One to Many	5	0.779	0.718
One to Many	10	0.803	0.747
One to Many	20	0.842	0.798
Numerar	5	0.513	0.371
Numerar	10	0.639	0.533
Numerar	20	0.684	0.592

Os resultados obtidos mostram que a estratégia One to Many é significativamente mais adequada para este problema de classificação multiclasse, atingindo desempenhos superiores de forma consistente em praticamente todas as arquiteturas testadas. Além disso, esta abordagem demonstrou maior estabilidade, apresentando melhorias mais graduais à medida que se aumentava a complexidade da rede. Por outro lado, a abordagem baseada na numeração da target, embora inicialmente apresente resultados bastante inferiores, revelou uma maior sensibilidade ao aumento da capacidade da rede e do número de epochs, registando melhorias mais acentuadas com arquiteturas mais profundas e mais tempo de treino. Este comportamento pode ser explicado pelo facto de esta estratégia exigir da rede uma aprendizagem mais precisa das relações entre os diferentes níveis de consumo. Por exemplo, uma determinada entrada pode ter Consumption MediumHigh (4) e a rede dar como output 3.4, que será arredondado para 3 (Medium), embora esteja muito perto do resultado correto. Isto fica ainda mais evidente se analisarmos a matriz de confusão dada pelo Scorer, correspondente à arquitetura de 4 layers (128, 64, 32, 16) com

5 epochs:

Prediction	Low	MediumLow	Medium	MediumHigh	High
Low	509	0	49	0	19
MediumLow	81	148	155	15	16
Medium	35	224	331	260	49
MediumHigh	8	124	85	345	87
High	0	7	0	6	67

O efeito referido é mais acentuado para Medium, onde a rede apresenta dificuldades em determinar a “fronteira” entre os diversos níveis médios de consumo.

De forma geral, os resultados evidenciam também que as redes neuronais desenvolvidas com Keras oferecem um grau de flexibilidade e configurabilidade substancialmente superior ao dos modelos anteriormente analisados, tornando a escolha da arquitetura, do método de representação dos dados e dos hiperparâmetros fatores decisivos para o desempenho final do modelo. Embora existisse margem para explorar arquiteturas mais complexas e processos de treino mais prolongados, essa exploração tornou-se limitada pelo elevado custo computacional associado ao treino destas redes.

Conclusão

Seguindo a metodologia SEMMA, começou-se por explorar o dataset de forma a identificar problemas de qualidade dos dados, padrões relevantes e possíveis transformações necessárias para a modelação. A análise realizada permitiu detectar valores em falta, outliers, inconsistências na target e features redundantes, conduzindo posteriormente à aplicação de diferentes estratégias de pré-processamento e transformação dos dados para resolver esses problemas.

Após a fase de modificação, foram desenvolvidos e avaliados cinco modelos de classificação distintos. De forma geral, os resultados mostram que os modelos baseados em árvores apresentaram o melhor desempenho neste problema, sobretudo os métodos de ensemble (Random Forest e Gradient Boosted Trees). Estes modelos revelaram também maior robustez relativamente às transformações aplicadas aos dados, sendo pouco sensíveis às diferentes estratégias de tratamento de missing values e outliers.

Por outro lado, as redes neuronais demonstraram maior dependência da preparação dos dados. Em particular, a escolha do método de normalização e da representação das variáveis categóricas teve impacto significativo no desempenho obtido. O modelo desenvolvido com Keras destacou-se pela maior flexibilidade e capacidade de adaptação da arquitetura da rede, permitindo explorar diferentes formas de representação da target e diferentes configurações de treino. Ainda assim, os resultados obtidos permaneceram inferiores aos melhores modelos baseados em árvores.

De forma global, conclui-se que este problema é mais adequado a modelos baseados em árvores, capazes de capturar relações complexas entre as variáveis sem exigir um pré-processamento tão sensível como o requerido pelas redes neuronais. Importa ainda referir que os resultados do modelo em Keras, ao contrário dos restantes modelos que foram avaliados com cross-validation, foram obtidos com uma simples partição hold-out, devido ao elevado custo computacional associado ao treino da rede neuronal.